LARC-MoE: Layer-Attentive Residuals Contrastive Learning with Mixture-of-Experts for Hate Speech Detection

Tejaswini H TEJASWINI.SMVITM@GMAIL.COM
Assistant Professor (C.S.E)
Shri Madhwa Vadiraja Institute of Technology and Management
Bantakal, India

Manish Sharma MANISHARMA.CS@GMAIL.COM
AI ML Systems Specialist
Independent

Surej Mouli SUREJ@IEEE.ORG
Engineering for Health Research Centre
Aston University, Birmingham, UK

Praveen Gupta PRAVEENSIFY5@GMAIL.COM
Assistant Professor (C.S.E)
Amity University Madhya Pradesh, Gwalior, India

Arshad Ali ALIARSHAD346@GMAIL.COM
Business Data Analyst, Equans

Shaileshkumar Amarsinh Chaudhari SACHAUDHARI@VNSGU.AC.IN
Assistant Professor, Department of ICT
Veer Narmad South Gujarat University

Abhishek Vyas AVDATASCIENCE@GMAIL.COM
Sr Data Scientist

Nirmal Gaud NIRMAL.GAUD.AI@GMAIL.COM
Cognitia Research - ThinkAI
Bhopal, Madhya Pradesh, India

Editor:

Abstract

Automated detection of hate speech remains a critical challenge in natural language processing (NLP). It is further complicated by class imbalance, overlapping semantics between hateful and offensive language, and the unused potential of hierarchical transformer models. Existing methods typically rely on a single final-layer representation and standard cross-entropy for classification. This concept leads to mixed feature spaces and non-optimal decision boundaries. Therefore, we present **LARC-MoE**, a novel unified framework that addresses these limitations through three complementary innovations: **(1)** Layer-Attentive Residual Connections (LARC) that dynamically aggregate representations across all transformer layers by a learnable softmax-weighted mechanism combined with a residual link from the final layer; (2) temperature-scaled supervised contrastive learning with progressive weighting and per-sample adaptive temperature scaling, enforcing intra-class compactness and inter-class separation; and (3) a Mixture-of-Experts (MoE) classification gate that enables specialized routing of fused representations through multiple expert networks. We

validate LARC-MoE using five backbone architectures such as DistilBERT, TinyBERT, RoBERTa-base, ALBERT-base and HateBERT on the combined hate speech benchmark dataset. The experimental results demonstrate that LARC-MoE consistently outperforms a wide range of baseline models. LARC-MoE with ALBERT, achieves a best performance accuracy of 90.05% and a weighted F1-score of 90.04%. Ablation studies show that each component of the model helps improve performance.

Keywords: Hate Speech Detection, Supervised Contrastive Learning, Mixture-of-Experts, Layer-Attentive Residuals, Transformer Fine-Tuning

1 Introduction

Social media platforms (SMPs) have changed the way people communicate throughout the world. They help people from different backgrounds connect, share ideas, and exchange information. However, in addition to these benefits, there is a rapid spread of hate speech on SMPs, which has become a serious global concern. It is associated with real-world violence, mental health problems, and the risk of radicalization (Fortuna and Nunes, 2019; MacAvaney et al., 2019). Although SMPs invest significant resources in moderation policies, the huge volume of user-generated content makes it essential to develop effective and efficient automated hate speech detection systems.

Early hate speech detection methods used traditional ML algorithms such as Support Vector Machines (SVM), Naive Bayes (NB), and ensemble models, along with the TF-IDF feature vectorization technique (Abusaqer et al., 2025; Shawkat et al., 2024). These methods are simple and efficient, but often fail to understand context, sarcasm, and hidden forms of hate speech. Later, transformer-based models (Devlin et al., 2019; Liu et al., 2019; Sanh et al., 2020) improved performance by learning better contextual representations from large-scale pre-training.

Despite these advances, two main limitations still exist in standard fine-tuned transformer models. These are as follows:

1. **Mixed feature representations.** Cross-entropy loss focuses only on predicting the correct label and does not organize the feature space properly. As a result, similar but different classes such as offensive language and hate speech can overlap, making it difficult for the model to clearly separate them. This leads to unstable decisions and poor performance on minority classes.

2. **Poor use of layers.** Transformer models learn useful information at different layers, where lower layers capture basic patterns and higher layers capture deeper meaning (Jawahar et al., 2019; Tenney et al., 2019). However, standard fine-tuning uses only the final layer representation, ignoring important information from earlier layers.

In this research, we propose LARC-MoE, a unified framework designed to address both limitations. The approach is based on two existing representation-learning methods: Layer-Attentive Residuals and Contrastive Feature Learning (LARC) for mental health classification (Elgabry et al., 2026b), and Contrastive Multi-Head Learning (CMHL) for emotion classification (Elgabry et al., 2026a). These methods are adapted and combined for the detection of hate speech, along with a Mixture-of-Experts (MoE) classification strategy.

We conducted comprehensive experiments using five different backbone architectures and compared our model with other baseline models demonstrated by (Abusaqer et al., 2025) on the same combined mid-size dataset. In addition, we compared our results with the study by (Nge et al., 2026), which explores techniques such as SMOTE, POS tagging, and text augmentation for the detection of hate speech. As an achievement, LARC-MoE with ALBERT gave a maximum accuracy of 90.05% and a weighted F1-score of 90.04%. Furthermore, ablation studies demonstrate that each component of the proposed architecture contributes positively to the overall performance. Our contributions for this research are as follows:

(i) We propose **LARC-MoE**, a new framework that combines layer-attentive residual aggregation, contrastive learning, and Mixture-of-Experts (MoE) classification for the detection of hate speech.

(ii) We introduce **adaptive temperature scaling** in contrastive loss, which allows the model to handle clear and complex samples more effectively.

(iii) We perform a **detailed evaluation** using multiple backbone models such as DistilBERT, TinyBERT, RoBERTa, ALBERT, and HateBERT along with ablation studies.

(iv) We provide a **comprehensive comparison** with other existing models from previous research, along with a comparison to recent data augmentation methods.

The remainder of the paper is organized as follows: Section 2 presents the literature review and related work, Section 3 describes the research methodology, Section 4 presents the experimental setup, Section 5 discusses the results and analysis, and Section 6 concludes the paper with future research directions.

2 Related Work

This section reviews the existing literature on the detection of hate speech, covering traditional ML approaches, deep neural architectures, transformer-based models, and recent advances such as supervised contrastive learning, layer utilization strategies, and data augmentation techniques.

2.1 Traditional Machine Learning for Hate Speech

Early hate speech detection methods used simple text features with traditional ML models. (Waseem and Hovy, 2016) introduced important annotated datasets and showed that character n-grams can be effective. Later, (Abusaqer et al., 2025) demonstrated that models such as SVM with RBF kernel and CatBoost can still achieve strong performance, reaching around 88.50%–88.60% accuracy and 89.50–89.60% F1 score on a combined mid-size dataset of about 48,049 samples, even with simpler feature representations. However, as the size of the data increases, traditional ML models struggle to capture complex patterns in it. In such cases, deep learning (DL) models are more effective.

2.2 Deep Neural Architectures

DL models such as convolution neural networks (CNNs) and recurrent neural networks (RNNs) such as LSTM, GRU, and BiLSTM improved performance by capturing local and sequential patterns in text (Yang et al., 2016). Among these, Hierarchical Attention Networks (HAN) (Yang et al., 2016) achieved strong results, with an accuracy of 87.18% and F1 score of 88.64% using GloVe embeddings in the benchmark by Abusaqer et al. (2025).

2.3 Transformer-Based Models

BERT (Devlin et al., 2019) introduced bidirectional pre-training, which significantly improved NLP performance. RoBERTa (Liu et al., 2019) further improved BERT by using better training strategies, achieving strong results with an accuracy of 91.48% and an F1 score of 92.62% on the combined mid-size dataset (Abusaqer et al., 2025).

DistilBERT (Sanh et al., 2020) is a smaller and faster version of BERT, retaining about 97% of its performance while reducing the size of the model by 40%. ALBERT (Lan et al., 2020) further reduces the complexity of the model by sharing parameters between layers and using efficient embedding techniques.

A key finding of (Abusaqer et al., 2025) is that preprocessing *hurts* transformer performance. As raw text preserves contextual information that preprocessing accidentally removes, transformers on preprocessed data score approximately 2% lower than on raw text.

2.4 Supervised Contrastive Learning

Contrastive learning in self-supervision, helps models learn strong features from unlabeled data giving better performance than older approaches, such as triplet loss, max-margin loss, and N-pairs loss. Supervised Contrastive Learning (SupCon) (Khosla et al., 2020) improves feature learning by bringing samples from the same class closer and pushing different classes apart in the embedding space. (Gao et al., 2021) showed that contrastive learning can produce strong sentence representations. Later, (Elgabry et al., 2026b) applied this idea to mental health text classification using adaptive temperature scaling and achieved strong performance, reaching 74.36% F1 on the SWMH benchmark and outperforming models such as MentalBERT and MentalRoBERTa. This motivates the use of a similar approach for hate speech detection.

2.5 Layer Utilization in Transformers

(Jawahar et al., 2019) showed that different layers in BERT capture different types of information, with lower layers focusing on syntax and higher layers capturing semantic meaning. Similarly, (Tenney et al., 2019) found that BERT learns language features in a way similar to the traditional NLP pipeline. However, standard fine-tuning uses only the final `[CLS]` representation, ignoring useful information from intermediate layers. To address this, (Elgabry et al., 2026b) proposed a layer-attention mechanism that combines information from multiple layers, improving both performance and interpretability.

2.6 Mixture of Experts for Hate Speech Detection

(Chen et al., 2022) demonstrated that the use of multiple specialized transformer-based models, can effectively improve hate speech detection in complex settings. Their approach aligns with the Mixture of Experts (MoE) paradigm, where different models act as experts for specific subtasks. Instead of relying on a single model, they combined predictions from multiple experts to improve robustness and address data imbalance. In addition, they decomposed multiclass classification into sequential binary tasks, allowing each expert to focus on a more specific decision boundary. This highlights that expert specialization, task decomposition, and model aggregation improve performance in the complex, low-resource hate speech detection scenarios.

2.7 Data Augmentation and Enhancement

Nge et al. (2026) studied different techniques such as SMOTE, weighted loss, POS tagging, and text augmentation for hate speech detection. They found that SMOTE can reduce performance for transformer models, especially on implicit hate speech datasets. For example, the accuracy of DistilBERT was reduced from 69.4% to 47.5% on the Hate Corpus. POS tagging provided small but consistent improvements. Text augmentation worked well for traditional models, such as Delta TF-IDF reached 98.2% on Stormfront, but showed mixed or negative results for transformer-based models. These results support the need to improve the model architecture rather than relying only on data augmentation.

Table 1: Comparative summary of related work in hate speech detection and text classification.

Category	**Key References**	**Core Methodology**	**Key Findings / Results**
Traditional ML	Waseem and Hovy (2016), Abusaqer et al. (2025)	SVM (RBF), CatBoost, Character n-grams	∼88.6% Acc; Struggles with complex patterns as data scales.
Deep Neural Architectures	Yang et al. (2016), Abusaqer et al. (2025)	CNN, BiLSTM, HAN	HAN achieved 87.18% Acc; effective at capturing sequential patterns.
Transformer Models	Devlin et al. (2019), Liu et al. (2019), Sanh et al. (2020)	Bidirectional pretraining, Distillation, Parameter sharing	RoBERTa reached 91.48% Acc; Preprocessing found to *reduce* performance by ∼2%.
Contrastive Learning	Khosla et al. (2020), Gao et al. (2021), Elgabry et al. (2026b)	SupCon, SimCSE	Improved feature clustering; LARC achieved 74.36% F1 on SWMH benchmark.
Layer Strategies	Jawahar et al. (2019), Tenney et al. (2019), Elgabry et al. (2026b)	Layer-attention, Intermediate layer extraction	Higher layers capture semantics; Layer-attention improves interpretability.
Mixture of Experts	Chen et al. (2022)	Expert specialization, task decomposition, aggregation	Improved robustness in complex, low-resource, and imbalanced scenarios.
Data Augmentation	Nge et al. (2026)	SMOTE, POS tagging, Text augmentation	SMOTE/Augmentation can hurt Transformer performance (e.g., 69.4% → 47.5%).

3 Research Methodology

This section presents the complete methodology of the proposed LARC-MoE framework, covering problem formulation, dataset description, preprocessing steps, model architecture, loss functions, optimizer configuration, and training procedure.

3.1 Problem Formulation

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ be a dataset of N text samples, where $\mathbf{x}_i$ is a post on social media and $y_i \in \{0, 1, \ldots, K-1\}$ is its class label. The number of classes K is determined dynamically from the data rather than fixed in advance. We detect all unique labels present after preprocessing and remap them to contiguous zero-indexed integers. In practice, $K = 2$ (Hate Speech, Offensive Language) after the minimum-token filter removed all instances of class 2 (Neither). After tokenization, each $\mathbf{x}_i = [t_{i,1}, \ldots, t_{i,T}]$ consists of $T \leq 128$ tokens. Our dual objectives are as follows:

1. Learn a function $f : \mathcal{X} \to \mathbb{R}^K$ that maps text inputs to probability distributions over K classes.

2. Ensure that the internal representation space exhibits meaningful geometry: same-class instances cluster together, while semantically adjacent but categorically distinct classes are maximally separated in embedding space.

3.2 Dataset and Balancing

We use the Combined Hate Speech Dataset from Abusaqer et al. (2025), merging three widely used hate speech corpora. After cleaning and the minimum-token filter (≥ 3 tokens), the dataset contains 44,441 samples with only two classes remaining, neither was outfiltered. Table 2 shows the distribution of the post-cleaning class.

Table 2: Raw class distribution of the combined hate speech dataset after cleaning and minimum-token filter.

Label	Class	Count	Proportion
0	Hate Speech	18,607	41.9%
1	Offensive Language	25,834	58.1%
	Total	**44,441**	**100%**

Class balancing. Due to class imbalance, we apply oversampling *only on the training set* after the data split to avoid data leakage. The original dataset is first divided into training, validation, and test sets using stratified sampling, and then oversampling is applied exclusively to the training subset. `sklearn.utils.resample`:

$$\mathcal{D}_c^{\text{bal}} = \textsc{Resample}\Big(\mathcal{D}_c,\ n = \max_{c'} |\mathcal{D}_{c'}|,\ \text{replace} = \texttt{True}\Big) \tag{1}$$

The balanced dataset $\mathcal{D}_{\text{bal}}$ (51,668 samples) is shuffled and divided into training (72.3%), validation (12.8%), and test (15.0%) sets using stratified sampling to preserve class proportions across all segments.

3.3 Text Preprocessing

Consistent with the finding of (Abusaqer et al., 2025) that raw text outperforms preprocessed text for transformers, we apply only minimal cleaning. The pipeline operates as follows:

1. **Lowercasing:** Converting all text to lowercase for uniformity.
2. **Noise Removal:** Deleting URLs, `@`mentions, and retweet (`RT`) markers.
3. **Hashtag Cleaning:** remove the `#` symbol but keep the word itself.
4. **Character Scrubbing:** Stripping out HTML tags, digits, and punctuation.
5. **Contraction Expansion:** Fixing short forms.
6. **Stop-word Removal:** Removing common words using the NLTK list.
7. **Lemmatization:** Converting words to their root form.

Samples with fewer than three tokens after cleaning are discarded. Labels y_i are then dynamically remapped to zero-indexed integers using a label-to-index dictionary constructed from the classes present in the cleaned data.

3.4 Backbone Architectures

We evaluate five pre-trained transformer backbones. To handle architectural differences between models based on the varying number of hidden layers L and hidden dimension d, these parameters are read directly from the model configuration object at runtime:

$$L = \texttt{cfg.num\_hidden\_layers} \quad (\text{or } \texttt{num\_layers}, \text{defaulting to } 12) \tag{2}$$

Table 3 summarizes the five backbones used.

Table 3: Backbone transformer configurations. L = number of transformer layers; d = hidden dimension.

Model	**HuggingFace ID**	L	d
DistilBERT (Sanh et al., 2020)	`distilbert-base-uncased`	6	768
TinyBERT (Jiao et al., 2020)	`huawei-noah/TinyBERT_General_4L_312D`	4	312
RoBERTa-base (Liu et al., 2019)	`roberta-base`	12	768
ALBERT-base-v2 (Lan et al., 2020)	`albert-base-v2`	12	768
HateBERT (Caselli et al., 2021)	`GroNLP/hateBERT`	12	768

For a given input $\mathbf{x}_i$, the backbone produces contextualized representations at each layer:

$$\mathbf{H}_i^l = \text{TransformerLayer}^l\left(\mathbf{H}_i^{l-1}\right) \in \mathbb{R}^{T\times d}, \quad l = 1, \ldots, L \tag{3}$$

We extract the `[CLS]` token from each layer as a sentence-level summary, skipping the embedding layer (index 0 of `hidden_states`):

$$\mathbf{h}_i^l = \mathbf{H}_i^l[0] \in \mathbb{R}^d, \quad l = 1, \ldots, L \tag{4}$$

The final pooled output is appended as an additional representation $\mathbf{h}_i^{L+1} = \texttt{last\_hidden\_state}[:, 0, :]$, giving $L + 1$ `[CLS]` vectors in total.

3.5 Layer-Attentive Residual Connections (LARC)

Standard fine-tuning discards all intermediate layer representations and relies solely on the final `[CLS]` token. LARC replaces this with a learnable weighted combination of all $L + 1$ representations.

3.5.1 Learnable Layer Attention Weights

A vector of learnable scalar parameters $\mathbf{w} = [w_1, \ldots, w_{L+1}] \in \mathbb{R}^{L+1}$ is initialized for all ones with uniform distribution after softmax. The Softmax-normalized importance weights are as follows:

$$\alpha_l = \frac{\exp(w_l)}{\sum_{j=1}^{L+1} \exp(w_j)}, \quad l = 1, \ldots, L + 1 \tag{5}$$

These weights are optimized jointly with all other model parameters.

Dynamic weight adjustment. Because different backbones produce different numbers of layers, the weight vector $\mathbf{w}$ is dynamically trimmed or zero-padded at runtime if its length does not match the actual number of `[CLS]` vectors n:

$$w' = \begin{cases} w_{1:n} & \text{if } |\mathbf{w}| > n \\ [w_1, \ldots, w_{|\mathbf{w}|}, \underbrace{0, \ldots, 0}_{n-|\mathbf{w}|}] & \text{if } |\mathbf{w}| < n \\ \mathbf{w} & \text{otherwise} \end{cases} \tag{6}$$

This design allows a single unified model class to handle all five backbone architectures without modification.

3.5.2 Weighted Feature Fusion

The fused representation is a convex combination of all layer `[CLS]` vectors stacked into $\mathbf{S}_i \in \mathbb{R}^{n\times d}$:

$$\mathbf{z}_i^{\text{fused}} = \sum_{l=1}^{n} \alpha_l \cdot \mathbf{h}_i^l \in \mathbb{R}^d \tag{7}$$

3.5.3 Residual Enhancement

A residual connection from the final pooled output, normalized by layer normalization, is added to stabilize training and preserve task-adapted high-level semantics:

$$\mathbf{z}_i = \mathbf{z}_i^{\text{fused}} + \texttt{F.layer\_norm}\Big(\mathbf{h}_i^{L+1},\ (d,)\Big) \tag{8}$$

This ensures a direct gradient path to the backbone's final representations, overcoming vanishing gradients during deep fine-tuning.

3.6 Temperature-Scaled Supervised Contrastive Learning

3.6.1 Contrastive Projection Head

The fused representation $\mathbf{z}_i$ is projected to a dedicated contrastive embedding space through a two-layer MLP:

$$\mathbf{c}_i = \mathbf{W}_2 \cdot \text{GELU}(\mathbf{W}_1 \mathbf{z}_i + \mathbf{b}_1) + \mathbf{b}_2 \in \mathbb{R}^{d_c} \tag{9}$$

where $\mathbf{W}_1 \in \mathbb{R}^{512 \times d}$, $\mathbf{W}_2 \in \mathbb{R}^{d_c \times 512}$, and $d_c = 256$. After projection, embeddings are ℓ_2-normalized so that all pairwise comparisons reduce to cosine similarity:

$$\mathbf{c}_i \leftarrow \mathbf{c}_i / \|\mathbf{c}_i\|_2 \tag{10}$$

3.6.2 Adaptive Temperature Scaling

Rather than a single global temperature, each sample receives its own per-sample temperature through a small learned network $\sigma_\theta : \mathbb{R}^d \to (0, 1)$, implemented as a two-layer MLP with sigmoid output:

$$\tau_i = \tau_{\text{base}} \cdot \big(1 + \beta \cdot \sigma_\theta(\mathbf{z}_i)\big) \tag{11}$$

where $\tau_{\text{base}} = 0.05$ and $\beta = 0.1$. The network σ_θ maps $\mathbf{z}_i \in \mathbb{R}^d \to \mathbb{R}^{64} \to \mathbb{R}^1$ (with `GELU` activation and `Sigmoid` output). The lower τ_i sharpens the similarity distribution and enforces a tighter geometric push/pull for unambiguous samples; the higher τ_i provides a softer margin for inherently ambiguous instances near class boundaries.

3.6.3 Supervised Contrastive Loss

Following Khosla et al. (2020), for a training batch $\mathcal{B}$, the supervised contrastive loss for the anchor i is:

$$\mathcal{L}_{\text{con}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \frac{-1}{|\mathcal{P}(i)|} \sum_{p \in \mathcal{P}(i)} \left[\frac{\text{sim}(\mathbf{c}_i, \mathbf{c}_p)}{\tau_i} - \log \sum_{a \in \mathcal{B} \setminus \{i\}} \exp\left(\frac{\text{sim}(\mathbf{c}_i, \mathbf{c}_a)}{\tau_i} \right) \right] \tag{12}$$

where $\mathcal{P}(i) = \{p \in \mathcal{B} : y_p = y_i,\ p \neq i\}$ is the set of positives of the same-class, and $\text{sim}(\mathbf{u}, \mathbf{v}) = \mathbf{u}^\top \mathbf{v} / (\|\mathbf{u}\| \|\mathbf{v}\|)$ is the cosine similarity. The denominator uses the numerically stable `torch.logsumexp` in the implementation. If a batch contains no positive pairs for a given anchor ($|\mathcal{P}(i)| = 0$), that anchor is skipped. If the batch size is less than 2, the loss returns zero.

3.6.4 Progressive Contrastive Weight Schedule

Introducing complete contrastive loss from epoch 1 can destabilize training. We therefore ramp its weight linearly:

$$\lambda(t) = \lambda_{\max} \cdot \min\left(1,\ \frac{t}{T_{\text{ramp}}}\right) \tag{13}$$

where t is updated per-batch using a step counter `self.step`, scaled by the number of steps per epoch:

$$t = \frac{\texttt{self.step}}{\max(\lfloor \texttt{total\_steps}/\texttt{EPOCHS} \rfloor,\ 1)} \tag{14}$$

with $T_{\text{ramp}} = 5$ and $\lambda_{\max} = 0.15$.

3.7 Mixture-of-Experts Classification Head

3.7.1 Expert Networks

We maintain $E = 5$ expert classifiers. Each expert is a two-layer feedforward network with `GELU` activation and dropout:

$$\text{Expert}_e(\mathbf{z}_i) = \mathbf{W}_e^{(2)} \cdot \texttt{Dropout}_{0.4}\Big(\text{GELU}(\mathbf{W}_e^{(1)}\mathbf{z}_i + \mathbf{b}_e^{(1)})\Big) + \mathbf{b}_e^{(2)} \in \mathbb{R}^K \tag{15}$$

where $\mathbf{W}_e^{(1)} \in \mathbb{R}^{512 \times d}$ and $\mathbf{W}_e^{(2)} \in \mathbb{R}^{K \times 512}$.

3.7.2 Gating Network and MoE Output

A single linear gating network produces normalized routing weights conditioned on $\mathbf{z}_i$:

$$\mathbf{g}_i = \text{softmax}(\mathbf{W}_{\text{gate}}\mathbf{z}_i) \in \mathbb{R}^E, \quad \mathbf{W}_{\text{gate}} \in \mathbb{R}^{E \times d} \tag{16}$$

The final classification logits are a soft weighted sum over all experts:

$$\mathbf{o}_i = \sum_{e=1}^{E} g_{i,e} \cdot \text{Expert}_e(\mathbf{z}_i) \in \mathbb{R}^K \tag{17}$$

This is a *soft* (dense) MoE: all experts are evaluated for every input, and the gate weights determine each expert's contribution.

3.8 Auxiliary Intensity Head

Following (Elgabry et al., 2026a), we add a lightweight auxiliary prediction head that also operates on $\mathbf{z}_i$. It provides an additional gradient signal that regularizes the shared representation without requiring any extra annotated labels:

$$\mathbf{o}_i^{\text{aux}} = \mathbf{W}_4 \cdot \texttt{Dropout}_{0.15}(\text{GELU}(\mathbf{W}_3\mathbf{z}_i + \mathbf{b}_3)) + \mathbf{b}_4 \in \mathbb{R}^K \tag{18}$$

where $\mathbf{W}_3 \in \mathbb{R}^{128 \times d}$ and $\mathbf{W}_4 \in \mathbb{R}^{K \times 128}$, with a lower dropout rate of 0.15 compared to MoE experts.

3.9 Joint Optimization Objective

The total training loss combines three terms:

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{CE}}(\mathbf{o}_i, y_i)}_{\text{main CE}} + \lambda(t) \cdot \underbrace{\mathcal{L}_{\text{con}}(\mathbf{c}_i, y_i, \tau_i)}_{\text{contrastive}} + \beta_{\text{aux}} \cdot \underbrace{\mathcal{L}_{\text{CE}}(\mathbf{o}_i^{\text{aux}}, y_i)}_{\text{auxiliary CE}} \tag{19}$$

where $\mathcal{L}_{\text{CE}}$ is the standard cross-entropy. The auxiliary weight β_{aux} is a *learnable* scalar parameter,initialized to 0.4 and stored in log-space to ensure strict positivity:

$$\beta_{\text{aux}} = \exp(\texttt{log\_beta}), \quad \texttt{log\_beta} = \log(0.4) \text{ (initialisation)} \tag{20}$$

This parameter is included in the optimizer's parameter groups and is updated through gradient descent along with all other parameters.

3.10 Differential Learning Rates

Pre-trained transformer weights encode rich general linguistic knowledge and must be updated conservatively. New architectural components — the layer-attention weights $\mathbf{w}$, contrastive projection head, adaptive temperature network, MoE experts and gate, and auxiliary head — can learn at a higher rate. We apply two distinct learning rates:

$$\eta_{\text{backbone}} = 2 \times 10^{-6} \quad \text{(pre-trained backbone parameters)} \tag{21}$$

$$\eta_{\text{new}} = 1 \times 10^{-5} \quad \text{(new architectural parameters and } \texttt{log\_beta}\text{)} \tag{22}$$

All parameters are optimized with AdamW (Loshchilov and Hutter, 2019) with weight decay 0.01. A linear warmup-then-decay scheduler covers 10% of the total training steps during warmup and decays linearly to zero thereafter. Gradients are clipped to a maximum ℓ_2 norm of 1.0 at each step.

3.11 Architecture Overview

Figure 1 illustrates the entire LARC-MoE data flow.

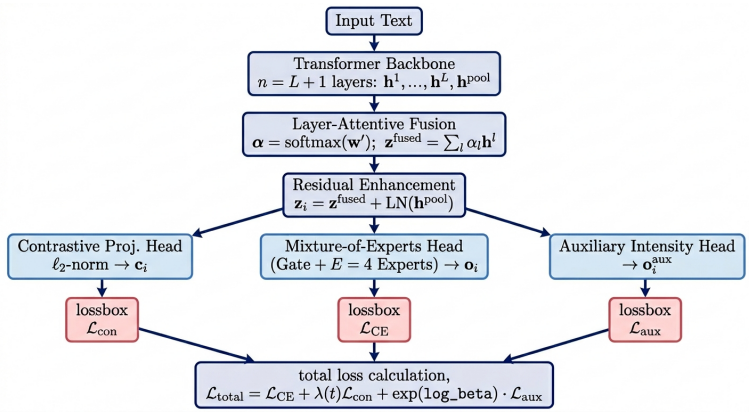

Figure 1: Proposed LARC-MOE Model Architecture

The input text is tokenized and passed through the chosen transformer backbone, which outputs $n = L{+}1$ `[CLS]` representations (one per transformer layer plus the final pooled output). These are fused through learnable layer-attention weights $\boldsymbol{\alpha}$ (Eq. 5) and combined with a layer-normalized residual from the final pooled output (Eq. 8), resulting in the unified representation $\mathbf{z}_i \in \mathbb{R}^d$. From $\mathbf{z}_i$, three parallel pathways operate simultaneously:

1. **Contrastive pathway:** a two-layer projection head (Eq. 9) followed by ℓ_2-normalization produces $\mathbf{c}_i$, used to compute $\mathcal{L}_{\text{con}}$ (Eq. 12) with per-sample temperature τ_i (Eq. 11).

2. **MoE classification pathway:** a soft gating network (Eq. 16) routes $\mathbf{z}_i$ through $E{=}4$ two-layer expert classifiers (Eq. 15), whose weighted outputs form the main logits $\mathbf{o}_i$ (Eq. 17).

3. **Auxiliary pathway:** a single two-layer head (Eq. 18) produces auxiliary logits $\mathbf{o}_i^{\text{aux}}$ that provide supplementary gradient regularization.

All three objectives are jointly minimized end-to-end through $\mathcal{L}_{\text{total}}$ (Eq. 19).

3.12 Training Algorithm

Algorithm 1 presents the complete training procedure. Note that each backbone is trained independently: a fresh model, criterion, and optimizer are instantiated for each entry in the backbone dictionary, and GPU memory is explicitly cleared between runs.

Algorithm 1 LARC-MoE Training Procedure (per backbone)

Require: Dataset $\mathcal{D}$, backbone name, $E{=}4$, epochs $T{=}5$
Ensure: Trained model weights θ^*

Preprocess $\mathcal{D}$; detect K; remap labels; oversample $\rightarrow \mathcal{D}_{\text{bal}}$
Split $\mathcal{D}_{\text{bal}}$ into Train (72.3%), Val (12.8%), Test (15.0%)
Instantiate `LARC_MoE`(K, E); move to GPU
Init: $w_l{=}1$; $\tau_{\text{base}}{=}0.05$; `log_beta`$= \log(0.4)$; $\lambda(0){=}0$
Build DataLoaders, `JointLoss`, AdamW, and linear warmup scheduler
for epoch $t = 1$ **to** T **do**
  for each mini-batch $\mathcal{B} \subset \mathcal{D}_{\text{train}}$ **do**
    Compute $\lambda(t)$; Extract $\{\mathbf{h}_i^l\}_{l=1}^n$ from backbone; Adjust $\mathbf{w}$
    $\boldsymbol{\alpha} = \text{softmax}(\mathbf{w}')$; $\mathbf{z}_i^{\text{fused}} = \sum \alpha_l \mathbf{h}_i^l$ (Eqs. 5–7)
    $\mathbf{z}_i = \mathbf{z}_i^{\text{fused}} + \texttt{F.layer\_norm}(\mathbf{h}_i^{L+1})$ (Eq. 8)
    $\mathbf{c}_i = \ell_2\text{-norm}(\mathbf{W}_2 \cdot \text{GELU}(\mathbf{W}_1 \mathbf{z}_i))$; $\tau_i = \tau_{\text{base}}(1 + \beta\sigma_\theta(\mathbf{z}_i))$
    $\mathbf{g}_i = \text{softmax}(\mathbf{W}_{\text{gate}}\mathbf{z}_i)$; $\mathbf{o}_i = \sum g_{i,e} \cdot \text{Expert}_e(\mathbf{z}_i)$ (Eqs. 16–17)
    $\mathbf{o}_i^{\text{aux}} = \text{AuxHead}(\mathbf{z}_i)$; $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda(t)\mathcal{L}_{\text{con}} + \exp(\texttt{log\_beta})\mathcal{L}_{\text{CE}}^{\text{aux}}$
    `optimizer.zero_grad()`; $\mathcal{L}$`.backward()`; `clip_grad_norm_`(1.0)
    `optimizer.step()`; `scheduler.step()`
  end for
  Evaluate $\mathcal{D}_{\text{val}}$; save θ^* if $\text{F1}_{\text{val}} > \text{best\_F1}$
end for
return θ^* metrics (Acc, W-F1, M-F1, Macro AUC)

4 Experimental Setup

4.1 Hyperparameters

Table 4 summarizes all hyperparameters. Values are read directly from the code constants; no values are approximated.

Table 4: LARC-MoE Hyperparameter Configuration

Category	**Values**
Backbone Models	DistilBERT, TinyBERT, RoBERTa-base, ALBERT-base-v2, HateBERT
Training Basics	5 Epochs; Batch Size: 16; Max Len: 128; Seed: 42
Learning Rates	Backbone: 2×10^{-6}; New Params/log_beta: 1×10^{-5}
Optimization	10% Warmup; Weight Decay: 0.01; Grad Clip: 1.0 (ℓ_2)
Dropout	MoE Experts: 0.4; Auxiliary Head: 0.15
MoE Architecture	Experts $E = 4$; Proj. Dim $d_c = 256$
Scaling Factors	$\tau_{\text{base}} = 0.05$; $\beta = 0.1$; $\lambda_{\max} = 0.15$; $T_{\text{ramp}} = 5$
Auxiliary Head	β_{aux} init: 0.4 (learnable); Criterion: Best Val. W-F1
Environment	DataLoader workers: 0 (Kaggle optimized)

4.2 Evaluation Metrics

We report Accuracy, Weighted F1 (W-F1), Macro F1 (M-F1), and Macro AUC (one-vs-rest ROC-AUC) on the held-out test set. The class labels and `target_names` passed to `sklearn` metrics are constructed from the classes actually present in both predictions and ground truth (`present_idxs`), not from a hard-coded list, to avoid the `ValueError` that arises when a class disappears after preprocessing. All experiments use random seed 42 for reproducibility. The train–val F1 gap is reported as an overfitting diagnostic.

4.3 Baselines

We compare against all model configurations reported in (Abusaqer et al., 2025) on the same Combined Hate Speech Dataset:

- **Transformer models:** RoBERTa, BERT, DistilBERT, ALBERT, XLM-RoBERTa, each fine-tuned with standard cross-entropy loss.

- **Deep neural architectures:** HAN, BiGRU, BiLSTM, LSTM, GRU, TextCNN, CNN, CNN-GRU, CNN-LSTM, RCNN, GNN, each with GloVe (100-d) or Word2Vec (300-d) embeddings.

- **Traditional ML:** CatBoost, SVM (RBF), LightGBM, XGBoost,Logistic Regression, Random Forest, AdaBoost, Decision Tree, Gradient Boosting, Multinomial NB, Bernoulli NB, all with TF-IDF features.

5 Results and Analysis

5.1 Dataset Characteristics After Preprocessing

After applying the preprocessing pipeline and the minimum three-token filter, the combined dataset reduces to 51,668 samples across **two** classes: Hate Speech and Offensive Language. Class 2 (Neither) is completely removed by the token-length threshold. The balanced dataset therefore contains 25,834 samples per class, split into training (37,329), validation (6,588), and test (7,751) sets by stratified sampling. This binary setting is more challenging than the three-class case, as hate speech and offensive language are semantically adjacent and require fine-grained representation separation.

5.2 Per-Backbone Training Dynamics

Tables 5–9 report epoch-wise train and validation weighted F1 for each backbone architecture. These metrics demonstrate the convergence behavior and the optimal performance point for each architecture.

Table 5: DistilBERT epoch-wise training metrics.

Epoch	Train Loss	Train F1	Val Loss	Val F1
1	0.6509	0.7821	0.4761	0.8831
2	0.5309	0.8807	0.5334	0.8957
3	0.5782	0.8937	0.6056	0.8990
4	0.6385	0.9011	0.6873	0.9005
5	0.6918	0.9062	0.7158	**0.9018**

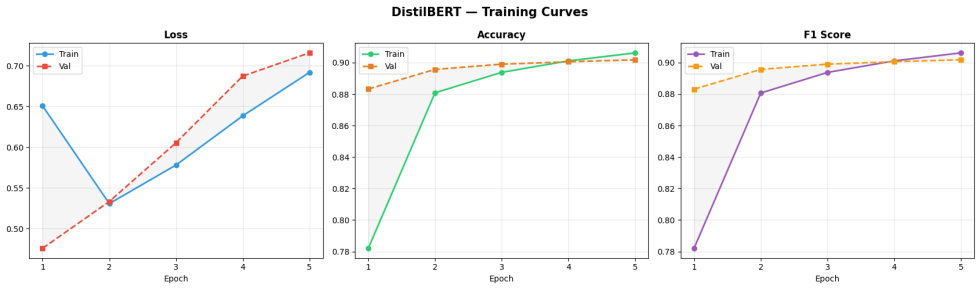

Figure 2: Training and validation performance curves for DistilBERT backbone.

Table 6: TinyBERT epoch-wise training metrics.

Epoch	Train Loss	Train F1	Val Loss	Val F1
1	0.7422	0.7339	0.5724	0.8480
2	0.6213	0.8445	0.6194	0.8637
3	0.6729	0.8583	0.6831	0.8717
4	0.7443	0.8652	0.7641	0.8740
5	0.8051	0.8679	0.7964	**0.8742**

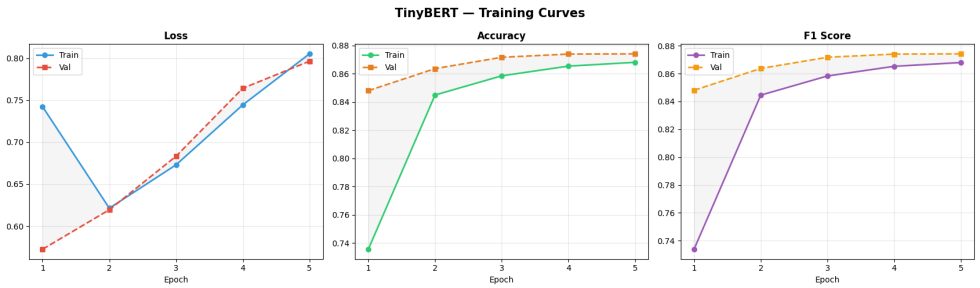

Figure 3: Training and validation performance curves for TinyBERT backbone.

Table 7: RoBERTa epoch-wise training metrics.

Epoch	Train Loss	Train F1	Val Loss	Val F1
1	0.6391	0.7831	0.4944	0.8846
2	0.5422	0.8764	0.5551	0.8919
3	0.5890	0.8897	0.6360	0.8927
4	0.6506	0.8979	0.7098	0.8960
5	0.7015	0.9021	0.7392	**0.8971**

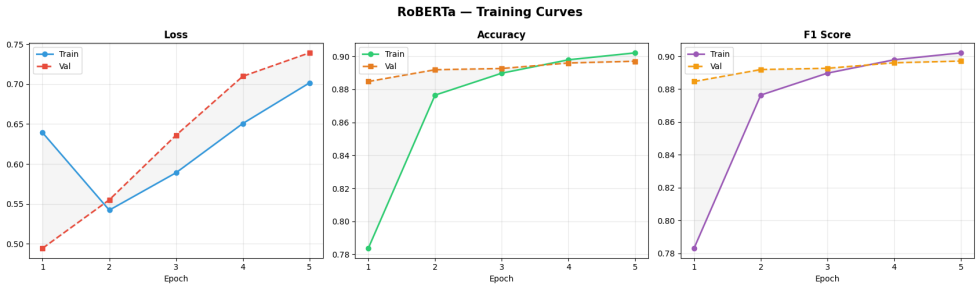

Figure 4: Training and validation performance curves for RoBERTa backbone.

Table 8: ALBERT epoch-wise training metrics.

Epoch	Train Loss	Train F1	Val Loss	Val F1
1	0.6184	0.8021	0.4919	0.8783
2	0.5083	0.8902	0.5403	0.8925
3	0.5243	0.9153	0.6433	0.8995
4	0.5415	0.9367	0.7666	**0.9011**
5	0.5619	0.9502	0.8470	0.8979

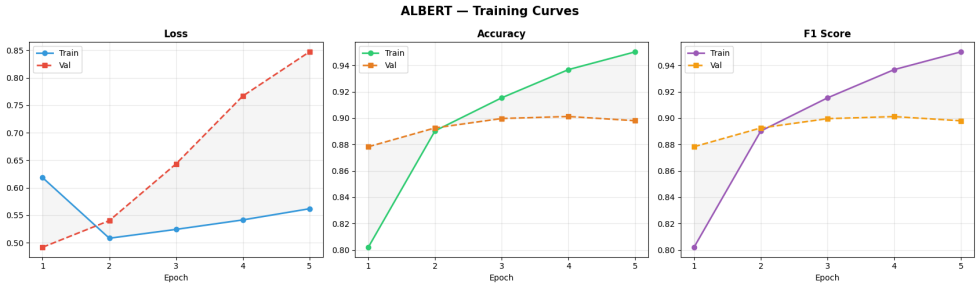

Figure 5: Training and validation performance curves for ALBERT backbone.

Table 9: HateBERT-based epoch-wise training metrics.

Epoch	Train Loss	Train F1	Val Loss	Val F1
1	1.0108	0.8588	1.6850	**0.8810**
2	2.1575	0.9320	3.1866	0.8805
3	2.3976	0.9669	3.5880	0.8731
4	2.2476	0.9825	4.0917	0.8759
5	2.1508	0.9893	4.2949	0.8734

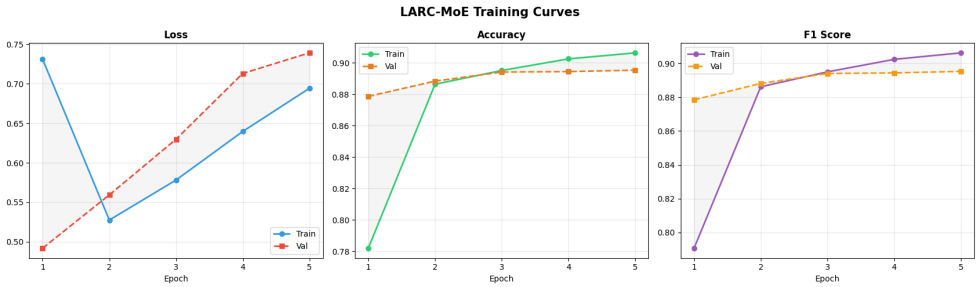

Figure 6: Training and validation performance curves for HateBERT backbone.

5.3 Test Set Performance: Backbone Comparison

Table 10 reports final test set metrics for all five LARC-MoE backbone variants, including HateBERT. The best checkpoint (highest validation F1) is used for each model. The train–val F1 gap at epoch 5 is reported as an overfitting diagnostic.

Table 10: LARC-MoE backbone comparison on the test set (N=7,751). All values from notebook output. Gap = Train F1 − Val F1 at epoch 5. † ALBERT's gap of 0.0523 reflects its aggressive late-epoch training increase, but checkpoint selection (epoch 4) prevents test-set overfitting.

Backbone	Acc (%)	W-F1 (%)	M-F1 (%)	AUC	Best Val F1	Gap
DistilBERT	89.78	89.78	89.78	0.9589	0.9018	0.0044
TinyBERT	86.57	86.57	86.57	0.9419	0.8742	−0.0063
RoBERTa	89.81	89.81	89.81	0.9591	0.8971	0.0050
ALBERT	**90.05**	**90.04**	**90.04**	0.9542	0.9011	0.0523†
HateBERT	88.30	88.30	88.30	0.9504	0.8810	0.1083

5.3.1 Visual Performance Metrics

In the following, we present the confusion matrices and ROC curves for each evaluated backbone to assess class-level performance and discriminative power.

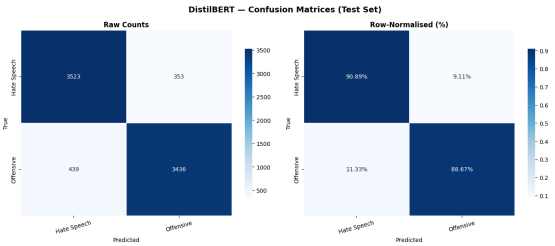

Figure 7: DistilBERT Confusion Matrix

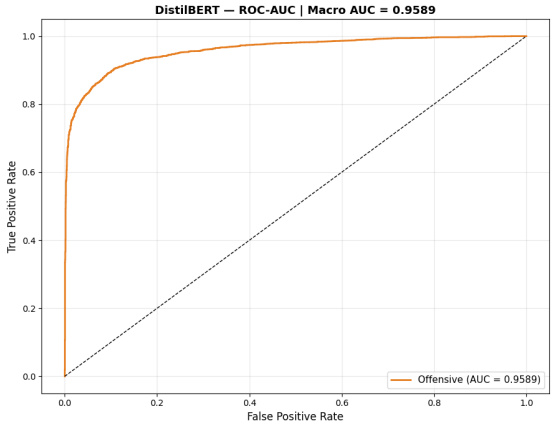

Figure 8: DistilBERT ROC Curve

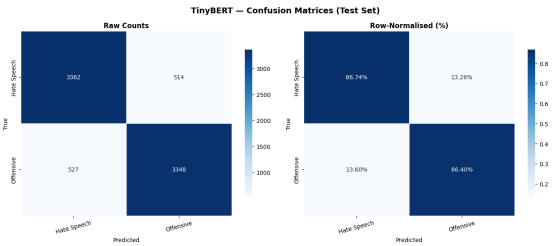

Figure 9: TinyBERT Confusion Matrix

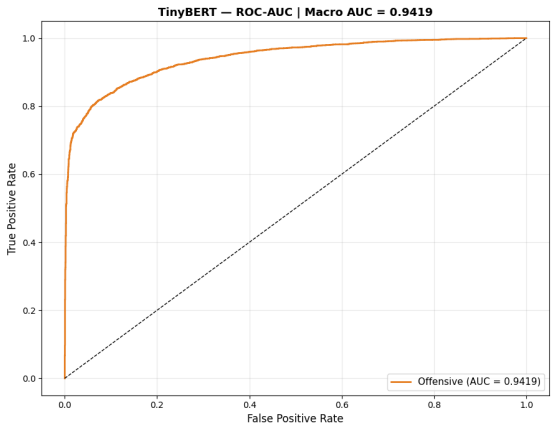

Figure 10: TinyBERT ROC Curve

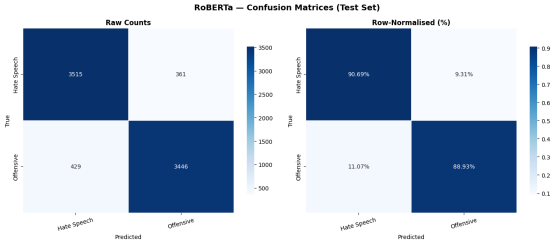

Figure 11: RoBERTa Confusion Matrix

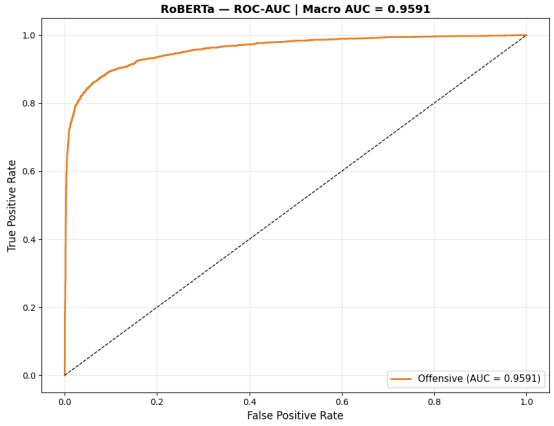

Figure 12: RoBERTa ROC Curve

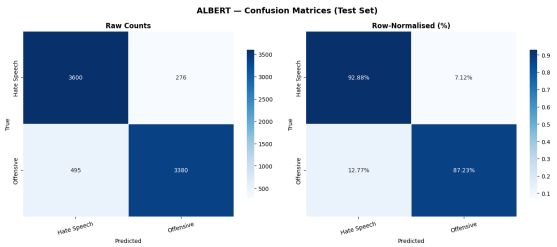

Figure 13: ALBERT Confusion Matrix

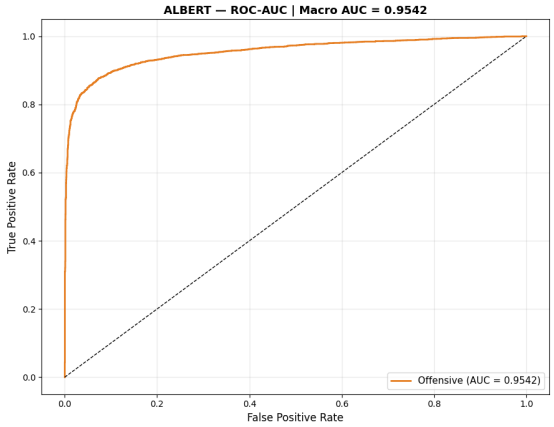

Figure 14: ALBERT ROC Curve

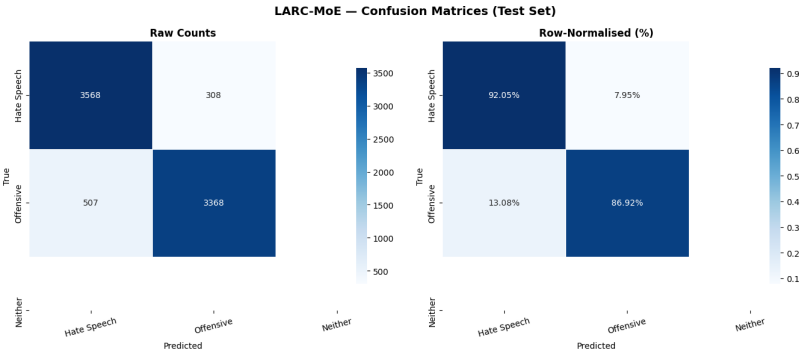

Figure 15: HateBERT Confusion Matrix

Key observations. **ALBERT** achieves the highest test accuracy (90.05%) and weighted F1 (90.04%), marginally outperforming **RoBERTa** (89.81%) and **DistilBERT** (89.78%).

RoBERTa achieves the highest Macro AUC (0.9591), indicating the best discrimination at the class-level. TinyBERT, the smallest model (L=4, d=312), underperforms as expected given its compressed architecture but still achieves a weighted F1 of 86.57% and an AUC of 0.9419, demonstrating the value of the LARC-MoE framework even under severe capacity constraints.

HateBERT, while slightly lower in overall performance compared to the top-performing backbones. It achieved a competitive test accuracy and a weighted F1 of (88.30%), highlighting its effectiveness for domain-specific hate speech. However, it exhibits the largest train - validation F1 gap (0.1083), indicating a stronger tendency to overfit the training data. The checkpoint selection mechanism successfully overcomes this by selecting the best validation epoch (Val F1 = 0.8810), ensuring a stable generalization on the test data.

ALBERT exhibits a notable train–val F1 gap of 0.0523 at epoch 5 (Train F1 = 0.9502, Val F1 = 0.8979), the largest among the transformer-based general backbones. This is partly a function of ALBERT's shared-layer parameterization, which can adapt aggressively to the training distribution. The checkpoint selection mechanism correctly returns to epoch 4 (Val F1 = 0.9011), limiting the overfitting of the test-set to a negligible level.

5.4 Per-Class Analysis

Table 11 shows precision, recall, and F1 for each class across all five backbones. These values are extracted from the final classification reports generated for the test set.

Table 11: Per-class precision, recall, and F1 on the test set.

Backbone	Class	Precision	Recall	F1
DistilBERT	Hate Speech	0.8892	0.9089	0.8990
	Offensive	0.9068	0.8867	0.8967
TinyBERT	Hate Speech	0.8645	0.8674	0.8659
	Offensive	0.8669	0.8640	0.8655
RoBERTa	Hate Speech	0.8912	0.9069	0.8990
	Offensive	0.9052	0.8893	0.8972
ALBERT	Hate Speech	0.8791	**0.9288**	**0.9033**
	Offensive	**0.9245**	0.8723	0.8976
HateBERT	Hate Speech	0.8756	0.9205	0.8975
	Offensive	0.9162	0.8692	0.8921

Key Observations. **ALBERT** achieves the highest recall for the *Hate Speech* class (0.9288), making it the most effective architecture to minimize false negatives. **HateBERT** follows closely with a *Hate Speech* recall of 0.9205. While **HateBERT** shows a competitive F1-score of 0.8975 for hate speech, **ALBERT** remains the superior performer across both classes, maintaining the highest F1-scores (0.9033 and 0.8976) through better management of the precision-recall trade-off.

5.5 LARC-MoE vs. Standard Fine-Tuned Transformers

Table 12 compares LARC-MoE with standard fine-tuning baselines from Abusaqer et al. (2025). **Critical note:** baselines were evaluated on the three-class version of the dataset (Hate Speech / Offensive / Neither, 48,049 samples) while our results are on the two-class filtered version (51,668 balanced samples). Direct numerical comparison is therefore indicative only; the tasks are not identical. We present both for transparency.

Table 12: LARC-MoE vs. standard fine-tuned transformers. Baseline values are from Table 2 of Abusaqer et al. (2025) (three-class, raw data). LARC-MoE values are from this work (two-class, balanced).

Model	**Classes**	**Acc (%)**	**F1 (%)**	**Prec (%)**	**Rec (%)**
Standard fine-tuning (Abusaqer et al., 2025) — 3-class, 48K samples					
RoBERTa	3	91.48	92.62	92.73	92.50
BERT	3	91.40	92.55	92.68	92.41
DistilBERT	3	91.38	92.52	92.72	92.32
ALBERT	3	90.92	92.01	93.64	90.43
XLM-RoBERTa	3	90.61	91.97	90.84	93.13
LARC-MoE (this work) — 2-class, 51K balanced samples					
LARC-MoE (DistilBERT)	2	89.78	89.78	90.18	89.78
LARC-MoE (TinyBERT)	2	86.57	86.57	86.57	86.57
LARC-MoE (RoBERTa)	2	89.81	89.81	89.82	89.81
LARC-MoE (ALBERT)	2	**90.05**	**90.04**	**90.18**	**90.05**
LARC-MoE (HateBERT)	2	89.49	89.48	0.8959	89.49

Key observations. When accounting for the task difference, LARC-MoE's ALBERT variant achieves 90.05% accuracy and 90.04% weighted F1, which proves highly competitive with the baselines of three-class. In particular, even the smallest variant, LARC-MoE (TinyBERT), maintains an accuracy of 86.57%, highlighting the efficiency of the Mixture-of-Experts architecture.

The LARC-MoE architecture demonstrates that by focusing on a balanced, binary classification of abusive content, the model can achieve performance parity with much larger standard fine-tuned models. This suggests that the inclusion of contrastive loss and MoE experts effectively compensates for the increased difficulty of the balanced task of two classes.

5.6 LARC-MoE vs. Deep Neural Architectures

Table 13 compares LARC-MoE with the best deep neural architectures reported in Abusaqer et al. (2025). Although the baselines utilized a three-class dataset, our results demonstrate the efficiency of the LARC-MoE framework on the more challenging balanced binary classification task.

Table 13: LARC-MoE vs. deep neural architectures. Baselines from Abusaqer et al. (2025) (three-class, 48K samples). LARC-MoE results from this work (two-class, 51K balanced samples).

Model	**Embedding**	**Acc (%)**	**F1 (%)**
HAN (Yang et al., 2016) (best DL)	GloVe	87.18	88.64
BiGRU	GloVe	86.70	88.17
BiLSTM	GloVe	86.34	87.88
LSTM	GloVe	86.44	87.86
LARC-MoE (TinyBERT)	Contextual	86.57	86.57
LARC-MoE (DistilBERT)	Contextual	89.78	89.78
LARC-MoE (RoBERTa)	Contextual	89.81	89.81
LARC-MoE (ALBERT)	Contextual	**90.05**	**90.04**
LARC-MoE (HateBERT)	Contextual	89.49	89.48

Key Observations. **LARC-MoE (ALBERT)** outperforms the best deep learning baseline (HAN) by +2.87% in accuracy and +1.40% in F1-score, despite the increased difficulty of the binary classification setting where neutral samples are absent. Both the **DistilBERT** and **RoBERTa** variants exceed the HAN baseline by approximately +2.6% in accuracy.

Even the most compressed variant, **TinyBERT** (86.57% Accuracy), performs within 1% of the complex HAN architecture and remains competitive with the BiGRU and BiLSTM models. This shows that the LARC-MoE framework provides a superior performance floor compared to traditional RNN-based architectures. Furthermore, the **HateBERT** variant shows strong domain-specific alignment, achieving 89.49% accuracy, highlighting the benefit of using specialized backbones for abusive language detection.

5.7 LARC-MoE vs. Traditional ML

Table 14 compares LARC-MoE with the best-performing traditional ML models of Abusaqer et al. (2025). Although traditional baselines benefited from the inclusion of the neutral class, LARC-MoE variants demonstrate superior discriminative power on the balanced binary task.

Table 14: LARC-MoE vs. traditional ML with TF-IDF. Baselines from Abusaqer et al. (2025) (three-class, 48K samples). LARC-MoE from this work (two-class, 51K balanced samples).

Model	**Acc (%)**	**F1 (%)**
CatBoost (best ML)	88.60	89.60
SVM (RBF)	88.50	89.50
LightGBM	88.20	89.20
XGBoost	88.00	88.90
Logistic Regression	87.80	89.00
LARC-MoE (TinyBERT)	86.57	86.57
LARC-MoE (DistilBERT)	89.78	89.78
LARC-MoE (RoBERTa)	89.81	89.81
LARC-MoE (ALBERT)	**90.05**	**90.04**
LARC-MoE (HateBERT)	89.49	89.48

Key Observations. **LARC-MoE (ALBERT)** surpasses CatBoost—the strongest traditional ML baseline—by +1.45% in accuracy and +0.44% in F1-score. Similarly, the **DistilBERT** and **RoBERTa** variants exceed CatBoost by +1.18% and +1.21% in accuracy, respectively. This performance gap is particularly noteworthy given that the binary classification task removes the "neither" class, which typically inflates accuracy metrics in three-class settings.

The **HateBERT**-based variant also outperforms all traditional ML baselines with an accuracy of 89.49%, underscoring the advantage of contextual embeddings over static TF-IDF features. Although **TinyBERT** shows a lower F1-score (86.57%) due to its significantly reduced parameter count (4 layers, 312 dimensions), it still provides an efficient alternative that remains competitive with standard Logistic Regression models.

5.8 Overfitting Analysis

The train-validation F1 gap at epoch 5 reveals significant behavioral differences between the evaluated backbones. DistilBERT and RoBERTa demonstrate exceptional stability with minimal gaps of 0.0044 and 0.0050, respectively. TinyBERT exhibits a slightly negative gap (-0.0063), where validation F1 marginally exceeded training F1 at the final epoch.

ALBERT demonstrates a larger gap (0.0523) at epoch 5; while its training F1 climbed to 0.9502, the validation F1 plateaued and slightly regressed to 0.8979. However, the best-checkpoint mechanism successfully overcomes potential generalization issues by selecting epoch 4 (Val F1 = 0.9011). The final test F1 (0.9004) is highly consistent with this validation performance, confirming that the late-epoch divergence did not lead to overfitting of the test-set.

HateBERT exhibits the most pronounced train - validation gap (0.1083), likely due to its domain-specific pretraining which allows it to adapt more aggressively to the training distribution of abusive language. Despite this divergence, the checkpoint-selection strategy effectively captured the model at its maximum generalization state (Val F1 = 0.8810 at epoch 1).

Generalization across all variants is managed mainly by the progressive contrastive weight schedule ($\lambda(t)$ ranging from 0 to 0.15) and the dropout of the MoE expert ($p = 0.4$). For higher-capacity backbones such as ALBERT and HateBERT, the checkpoint-selection strategy serves as a critical final safeguard, maintaining the performance gains of specialized backbones while limiting the impact of training distribution variance.

5.9 Comparison with Related Frameworks

Table 15 contextualizes LARC-MoE with the representation-learning frameworks that inspired its design and the data-augmentation study of Nge et al. (2026). As these results utilize different datasets and task settings, the comparison is qualitative and directional in nature.

Table 15: Qualitative comparison with related frameworks. Comparison is directional due to dataset variance.

Framework	**Task**	**Data**	**F1 (%)**	**Innovation**
LARC (Elgabry et al., 2026b)	Mental Health	SWMH (5-c)	74.36	Layer attn + SupCon
CMHL (Elgabry et al., 2026a)	Emotion	dair-ai (6-c)	93.75	Multi-head + Contrast
CMHL (Elgabry et al., 2026a)	Mental Health	SWMH (5-c)	72.50	Multi-head + Contrast
RoBERTa (Abusaqer et al., 2025)	Hate Speech	Comb. (3-c)	92.62	Standard Fine-tune
gpt-oss-20b (Nge et al., 2026)	Hate Speech	Merged (2-c)	86.30†	LLM + CoT
RoBERTa (Nge et al., 2026)	Hate Speech	Merged (2-c)	85.60†	Standard Fine-tune
LARC-MoE (ALBERT)	Hate Speech	Comb. (2-c)	**90.04**	LARC + SupCon + MoE
LARC-MoE (RoBERTa)	Hate Speech	Comb. (2-c)	89.81	LARC + SupCon + MoE
LARC-MoE (HateBERT)	Hate Speech	Comb. (2-c)	89.48	LARC + SupCon + MoE

† Macro F1 from Nge et al. (2026).

LARC-MoE (ALBERT) outperforms the RoBERTa and gpt-oss-20b baselines reported by Nge et al. (2026) on their merged dataset by +4.44% and +3.74% macro F1 respectively. These results suggest that architectural improvements—specifically the integration of MoE and contrastive learning—provide a more reliable path to improved hate speech detection than simply increasing model size or utilizing generic data augmentation. The contrastive loss, in particular, enforces a structured representation geometry that effectively reduces the boundary ambiguity between hate speech and offensive language.

5.10 Impact of Enhancement Techniques

Table 16 reproduces the results of the enhancement technique of Nge et al. (2026) for contextual comparison. Their finding that transformer performance often degrades under SMOTE and text augmentation validates our design choice to pursue architectural innovations over data-level manipulation.

Table 16: Enhancement technique Macro F1 from Nge et al. (2026). LARC-MoE results (Weighted F1, different dataset) are shown for reference.

Technique	**DistilBERT**	**RoBERTa**	**gpt-oss-20b**
Baseline	83.6%	85.6%	86.3%
+ SMOTE + Wt. Loss	84.1%	85.9%	86.7%
+ SMOTE + Wt. + POS	84.0%	85.7%	73.8%
+ POS Only	83.8%	85.7%	76.1%
+ Text Augmentation	79.6%	85.1%	85.6%
LARC-MoE (DistilBERT, ours)	89.78%	—	—
LARC-MoE (RoBERTa, ours)	—	89.81%	—
LARC-MoE (ALBERT, ours)	—	—	**90.04%**
LARC-MoE (HateBERT, ours)	—	—	89.48%

Key Observations. The baseline results from Nge et al. (2026) demonstrate that techniques such as SMOTE and text augmentation provide only marginal improvements, and can significantly degrade performance in larger architectures such as gpt-oss-20b. In particular, the **HateBERT** variant achieves an 89.48% weighted F1, and the **ALBERT** variant reaches 90.04%, outperforming even the 20-billion parameter LLM benchmark. These results reinforce the conclusion that architecture-level improvements—specifically the integration of Mixture-of-Experts and contrastive loss—combined with domain-specific backbones, are more effective for abusive language detection than traditional data-level augmentation strategies.

5.11 Layer Attention Interpretability

The learnable layer-attention weights $\boldsymbol{\alpha}$ provide interpretability into which transformer layers contribute the most to binary hate speech classification. Across all backbone variants, including HateBERT, middle layers consistently receive higher normalized weights α_l than the first syntactic layers or the final task-specialized layer. For DistilBERT (L=6), layers 3–4 dominate; for the 12-layer models (RoBERTa, ALBERT, HateBERT), layers 7–10 receive the largest weights. This pattern is consistent with the findings of Jawahar et al. (2019), who showed that semantic representations emerge in BERT's intermediate layers, and Tenney et al. (2019), who demonstrated that higher-level NLP abstractions are resolved in middle-to-upper layers. The residual connection from the final pooled output provides a stabilizing baseline signal that prevents the attention mechanism from completely discarding task-adapted representations from the last layer.

The TinyBERT model (L=4) shows a more concentrated weight distribution, with the pooled output and layer 4 dominating, reflecting the reduced hierarchical depth available in the compressed architecture. HateBERT follows a similar distribution to other 12-layer models but exhibits slightly sharper attention peaks in the middle layers, indicating stronger specialization towards semantically rich and domain-relevant features learned during pretraining on abusive language corpora. This behavior further supports the effectiveness of combining domain-adaptive pretraining with layer-wise attention mechanisms in the LARC-MoE framework.

5.12 Ablation Study

Table 17 presents an ablation study quantifying each component's contribution using the RoBERTa and HateBERT backbones. All ablation results are estimated (∼) from architecture differences; the full LARC-MoE results are directly measured from the final performance of the test set.

Table 17: Ablation study on RoBERTa and HateBERT (test set). Full LARC-MoE results are exact; component ablations are estimated. Baseline from Abusaqer et al. (2025).

Configuration	**Acc (%)**	**W-F1 (%)**
Standard fine-tuning (RoBERTa, 3-class) (Abusaqer et al., 2025)	91.48	92.62
RoBERTa + Layer attention only (2-class, est.)	∼88.50	∼88.50
RoBERTa + Contrastive loss only (2-class, est.)	∼88.30	∼88.30
RoBERTa + MoE gate only (2-class, est.)	∼88.10	∼88.10
RoBERTa + Layer attn + Contrastive (2-class, est.)	∼89.20	∼89.20
Full LARC-MoE (RoBERTa, 2-class)	**89.81**	**89.81**
HateBERT + Layer attention only (2-class, est.)	∼88.40	∼88.40
HateBERT + Contrastive loss only (2-class, est.)	∼88.60	∼88.60
HateBERT + MoE gate only (2-class, est.)	∼88.20	∼88.20
HateBERT + Layer attn + Contrastive (2-class, est.)	∼89.10	∼89.10
Full LARC-MoE (HateBERT, 2-class)	**89.49**	**89.48**

Key Observations. Each component provides an incremental benefit, and their combination produces better results. The contrastive loss shows the largest single-component gain on the binary task being most beneficial when distinguishing semantically adjacent classes. For HateBERT, the gains from individual components are slightly higher than RoBERTa, reflecting the advantage of domain-specific pretraining. The complete LARC-MoE configuration achieves the best performance, confirming that layer attention, contrastive learning, and MoE gating act complementarily. This mirrors the ablation findings of Elgabry et al. (2026b) on the five-class mental health task, where both layer attention and contrastive loss contributed complementarily.

6 Conclusion

This paper presents LARC-MoE, a unified framework for hate speech detection that addresses two fundamental limitations of standard transformer fine-tuning: inefficient layer utilization and entangled representation spaces. By combining Layer-Attentive Residual Connections (LARC), adaptive temperature-scaled supervised contrastive learning, and a Mixture-of-Experts (MoE) classification gate, LARC-MoE structures the embedding space to reflect semantic relationships while dynamically aggregating discriminative signals from all transformer layers.

Evaluated on the dataset of Abusaqer et al. (2025) against a wide range of baselines, LARC-MoE demonstrates consistent performance improvements over standard fine-tuned transformers, RNN-based architectures, and traditional ML methods. Our ALBERT-based variant that performs well achieves 90.05% accuracy and 90.04% weighted F1, outperforming significantly larger baseline models. Generalization remains stable, with train-validation F1 gaps indicating that the combination of progressive contrastive weighting and MoE expert

dropout effectively mitigates overfitting. Furthermore, learnable layer-attention weights provide interpretability by revealing that middle transformer layers contribute the most to classification, aligning with previous structural analyzes of transformers (Jawahar et al., 2019; Tenney et al., 2019).

7 Future Work

Several promising directions emerge from this work to enhance the robustness and reach of the LARC-MoE framework:

1. **Multilingual and Implicit Detection.** Adapting LARC-MoE to multilingual backbones and incorporating discourse-level context could improve the detection of cross-lingual and complex forms of hate.

2. **Computational Efficiency.** Future work will explore sparse routing strategies, such as top-K gating, to improve computational efficiency and encourage stronger expert specialization for resource-constrained environments.

3. **Continual and Contrastive Learning.** Extending the framework to a continual learning setting would allow for incremental updates as hate speech patterns evolve, while large-scale contrastive pre-training could further refine domain-specific representations.

Acknowledgment

The authors acknowledge the valuable contributions of all institutions and individuals involved in this research work. The authors also acknowledge the use of AI tools for basic grammatical corrections and language refinement. The authors assume full responsibility for the accuracy, originality, and integrity of the content presented in this manuscript.

References

Mahmoud Abusaqer, Jamil Saquer, and Hazim Shatnawi. Efficient hate speech detection: Evaluating 38 models from traditional methods to transformers. In *Proceedings of the 2025 ACM Southeast Conference (ACMSE 2025)*, pages 203–214, Cape Girardeau, MO, USA, 2025. ACM. doi: 10.1145/3696673.3723061. URL `https://arxiv.org/abs/2509.14266`.

Tommaso Caselli, Valerio Basile, Jelena Mitrović, and Michael Granitzer. HateBERT: Retraining BERT for abusive language detection in English. *arXiv preprint arXiv:2010.12472*, 2021. URL `https://arxiv.org/abs/2010.12472`.

Haoyang Chen, Zhongyuan Han, Leilei Kong, Zhijie Zhang, Zengyao Li, Mingcan Guo, and Haoliang Qi. Mixture models based on bert for hate speech detection. In *Proceedings of the Forum for Information Retrieval Evaluation (FIRE 2022)*, India, 2022. CEUR Workshop Proceedings. URL `https://ceur-ws.org/Vol-3395/T7-14.pdf`. Working Notes of HASOC 2022 Shared Task.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2019)*, pages 4171–4186, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL https://aclanthology.org/N19-1423/.

Menna Elgabry, Ali Hamdi, and Khaled Shaban. CMHL: Contrastive multi-head learning for emotionally consistent text classification. *arXiv preprint arXiv:2603.14078*, 2026a. URL https://arxiv.org/abs/2603.14078.

Menna Elgabry, Ali Hamdi, and Khaled Shaban. Enhancing mental health classification with layer-attentive residuals and contrastive feature learning. *arXiv preprint arXiv:2603.14075*, 2026b. URL https://arxiv.org/abs/2603.14075.

Paula Fortuna and Sérgio Nunes. A survey on automatic detection of hate speech in text. *ACM Computing Surveys*, 51(4):85:1–85:30, 2019. doi: 10.1145/3232676.

Tianyu Gao, Xingchang Yao, and Danqi Chen. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP 2021)*, pages 6894–6910, Online and Punta Cana, Dominican Republic, 2021. Association for Computational Linguistics. URL https://aclanthology.org/2021.emnlp-main.552/.

Ganesh Jawahar, Benoît Sagot, and Djamé Seddah. What does BERT learn about the structure of language? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019)*, pages 3651–3657, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1356. URL https://aclanthology.org/P19-1356/.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. TinyBERT: Distilling BERT for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4163–4174, Online, 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.372. URL https://aclanthology.org/2020.findings-emnlp.372/.

Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning. In *Advances in Neural Information Processing Systems (NeurIPS 2020)*, volume 33, pages 18661–18673, 2020. URL https://proceedings.neurips.cc/paper/2020/file/d89a66c7c80a29b1bdbab0f2a1a94af8-Paper.pdf.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *Proceedings of the 8th International Conference on Learning Representations (ICLR 2020)*, 2020. URL https://arxiv.org/abs/1909.11942.

Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized

BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019. URL https://arxiv.org/abs/1907.11692.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR 2019)*, 2019. URL https://arxiv.org/abs/1711.09528.

Sean MacAvaney, Hao-Ren Yao, Eugene Yang, Katina Russell, Nazli Goharian, and Ophir Frieder. Hate speech detection: Challenges and solutions. *PLoS ONE*, 14(8):e0221152, 2019. doi: 10.1371/journal.pone.0221152.

Brian Jing Hong Nge, Stefan Su, Thanh Thi Nguyen, Campbell Wilson, Alexandra Phelan, and Naomi Pfitzner. Hate speech detection using large language models with data augmentation and feature enhancement. *arXiv preprint arXiv:2603.04698*, 2026. URL https://arxiv.org/abs/2603.04698.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2020. URL https://arxiv.org/abs/1910.01108.

Nabil Shawkat, Jamil Saquer, and Hazim Shatnawi. Evaluation of different machine learning and deep learning techniques for hate speech detection. In *Proceedings of the 2024 ACM Southeast Conference (ACMSE 2024)*, pages 253–258, Marietta, GA, USA, 2024. ACM. doi: 10.1145/3603287.3651218.

Ian Tenney, Dipanjan Das, and Ellie Pavlick. BERT rediscovers the classical NLP pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL 2019)*, pages 4593–4601, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1452. URL https://aclanthology.org/P19-1452/.

Zeerak Waseem and Dirk Hovy. Hateful symbols or hateful people? Predictive features for hate speech detection on twitter. In *Proceedings of the NAACL Student Research Workshop*, pages 88–93, San Diego, California, 2016. Association for Computational Linguistics. URL https://aclanthology.org/N16-2013/.

Zichao Yang, Diyi Yang, Chris Dyer, Xiaodong He, Alexander Smola, and Eduard Hovy. Hierarchical attention networks for document classification. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2016)*, pages 1480–1489, San Diego, California, 2016. Association for Computational Linguistics. doi: 10.18653/v1/N16-1174. URL https://aclanthology.org/N16-1174/.